Java Coding Interview Question Bank

Deduplicated, ranked by importance and grouped by topic
Prepared for a Test Automation Engineer with 4+ years of experience

Questions in your original document	**304**
Exact duplicates removed (same question, different punctuation, case or formatting)	**9**
Near-duplicates merged (same question asked in different words)	**144**
Unique questions in this PDF	**151**

Section 1: Critical (Must Practise)	27 questions
Section 2: High Priority	63 questions
Section 3: Medium Priority	45 questions
Section 4: Low Priority / Quick Revision	16 questions

How to Use This Document

Programs are ordered by how likely they are to be asked and how much weight interviewers give them for a tester with 4+ years of experience. Work top to bottom: finish Section 1 before you spend time on Section 3 or 4. Inside each section the questions are grouped by topic, and within each topic the most important question comes first.

A red tag such as **[asked x3 in your bank]** means the same question showed up several times in your original document (in different wording). Repetition is a good signal that it is a frequently asked question.

Section	What it means
Critical (Must Practise) 27 questions	Asked in almost every coding round. Write each one by hand, in at least two ways (with and without built-in methods), and be ready to state time complexity and handle edge cases.
High Priority 63 questions	Frequent follow-ups and the next level of difficulty. Practise these once you can do Section 1 without hesitation, with focus on HashMap, Streams and file handling.
Medium Priority 45 questions	Occasionally asked, usually for SDET-leaning roles or as a challenge question. Know the approach even if you cannot write it perfectly.
Low Priority / Quick Revision 16 questions	Rarely asked for automation testers. Skim the idea, do not spend prep time here before Sections 1 to 3.

What Interviewers Look For (4+ Years)

Topic	What the interviewer is testing
String Programs	The most common live-coding round. They watch your approach (with and without built-ins), edge cases (null, empty, case) and complexity.
Number & Math Programs	Warm-up logic: loops, recursion, and clean handling of boundaries. Should be answerable in minutes.
Array Programs	Can you solve with and without extra collections, and explain time vs space trade-offs?
Collections & HashMap	Practical use of Map/Set/List: frequency counts, de-duplication, sorting maps, test-data handling.
Java 8 Streams	Modern, concise code: filter, map, collect, distinct, sorted, grouping.
Sorting & Searching	Basic algorithm knowledge and the ability to reason about complexity.
OOP, Design Pattern & Exception	Singleton, equals/hashCode, comparators and custom exceptions written correctly.
File Handling	Reading, writing and parsing files, which is common in test-data utilities.
Multithreading	Odd/even thread printing and deadlock are the usual pair; shows if you understand thread coordination.
Linked List, Stack & Matrix	Data-structure questions, seldom asked for testers but common for SDET-style roles.
Output & Concept Tricks	Short 'what will this print / is this allowed' questions.

Note: many entries were the same program phrased as 'Write a java code for...', 'Java program to...' or 'WAP to...'; they are merged into one task. One resume line ('Created a collection of employee records and sorted them by employee ID') was turned into a task. The 'Coding tasks:' prefix was dropped.

Table of Contents

Section 1: Critical (Must Practise) (27 questions) . . . 5
String Programs (10) . . . 5
Number & Math Programs (5) . . . 5
Array Programs (6) . . . 5
Collections & HashMap Programs (incl. Test Data Handling) (5) . . . 5
Sorting, Searching & Complexity (1) . . . 6

Section 2: High Priority (63 questions) . . . 7
String Programs (14) . . . 7
Number & Math Programs (11) . . . 7
Array Programs (13) . . . 7
Collections & HashMap Programs (incl. Test Data Handling) (8) . . . 8
Java 8 Streams Programs (5) . . . 8
Sorting, Searching & Complexity (2) . . . 8
OOP, Design Pattern & Exception Coding (7) . . . 8
File Handling Programs (1) . . . 9
Multithreading Programs (2) . . . 9

Section 3: Medium Priority (45 questions) . . . 10
String Programs (8) . . . 10
Number & Math Programs (4) . . . 10
Array Programs (7) . . . 10
Collections & HashMap Programs (incl. Test Data Handling) (4) . . . 10
Java 8 Streams Programs (3) . . . 10
Sorting, Searching & Complexity (2) . . . 11
OOP, Design Pattern & Exception Coding (1) . . . 11
File Handling Programs (4) . . . 11
Multithreading Programs (2) . . . 11
Linked List, Stack, Matrix & Advanced Problems (8) . . . 11
Output & Concept Trick Questions (2) . . . 11

Section 4: Low Priority / Quick Revision (16 questions) . . . 12
String Programs (4) . . . 12
Number & Math Programs (5) . . . 12
Array Programs (4) . . . 12
Sorting, Searching & Complexity (1) . . . 12
Linked List, Stack, Matrix & Advanced Problems (2) . . . 12

Topic Finder (all sections for one topic)

Use this if you want to study one topic at a time. Each number links to that topic's questions inside a section.

Topic	S1	S2	S3	S4	Total
String Programs	10 Qs, p.5	14 Qs, p.7	8 Qs, p.10	4 Qs, p.12	**36**
Number & Math Programs	5 Qs, p.5	11 Qs, p.7	4 Qs, p.10	5 Qs, p.12	**25**
Array Programs	6 Qs, p.5	13 Qs, p.7	7 Qs, p.10	4 Qs, p.12	**30**
Collections & HashMap Programs (incl. Test Data Handling)	5 Qs, p.5	8 Qs, p.8	4 Qs, p.10	-	**17**
Java 8 Streams Programs	-	5 Qs, p.8	3 Qs, p.10	-	**8**
Sorting, Searching & Complexity	1 Qs, p.6	2 Qs, p.8	2 Qs, p.11	1 Qs, p.12	**6**
OOP, Design Pattern & Exception Coding	-	7 Qs, p.8	1 Qs, p.11	-	**8**
File Handling Programs	-	1 Qs, p.9	4 Qs, p.11	-	**5**
Multithreading Programs	-	2 Qs, p.9	2 Qs, p.11	-	**4**
Linked List, Stack, Matrix & Advanced Problems	-	-	8 Qs, p.11	2 Qs, p.12	**10**
Output & Concept Trick Questions	-	-	2 Qs, p.11	-	**2**

S1 = Critical, S2 = High, S3 = Medium, S4 = Low priority.

Section 1: Critical (Must Practise) (27 questions)

Asked in almost every coding round. Write each one by hand, in at least two ways (with and without built-in methods), and be ready to state time complexity and handle edge cases.

String Programs (10)

1. Reverse a string (with and without built-in methods, using StringBuilder or a loop). **[asked x10 in your bank]**
2. Check whether a string is a palindrome. **[asked x6 in your bank]**
3. Check whether two strings are anagrams. **[asked x7 in your bank]**
4. Count the occurrences (frequency) of each character in a string. **[asked x6 in your bank]**
5. Find the duplicate characters in a string. **[asked x4 in your bank]**
6. Find the first (or last) non-repeating character in a string. **[asked x4 in your bank]**
7. Reverse the words (or each word) in a sentence, with or without split(). **[asked x6 in your bank]**
8. Count vowels and consonants (and digits and spaces) in a string. **[asked x6 in your bank]**
9. Remove duplicate characters from a string (print unique characters). **[asked x3 in your bank]**
10. Remove white spaces (and special characters) from a string, without built-in methods. **[asked x5 in your bank]**

Number & Math Programs (5)

11. Print the Fibonacci series (up to n terms or a given range). **[asked x7 in your bank]**
12. Find the factorial of a number (loop and recursion). **[asked x4 in your bank]**
13. Check whether a number is prime. **[asked x5 in your bank]**
14. Swap two numbers without a third variable. **[asked x3 in your bank]**
15. Check whether a number is a palindrome. **[asked x3 in your bank]**

Array Programs (6)

16. Find the largest and smallest (min and max) number in an array. **[asked x8 in your bank]**
17. Find the second largest (second highest) number in an array. **[asked x3 in your bank]**
18. Remove duplicates from an array. **[asked x2 in your bank]**
19. Find duplicate elements in an array. **[asked x4 in your bank]**
20. Find the missing number in an array (1 to n) or in a continuous sequence. **[asked x5 in your bank]**
21. Reverse an array.

Collections & HashMap Programs (incl. Test Data Handling) (5)

22. Remove duplicates from an ArrayList or List (using Set, keeping the order). **[asked x6 in your bank]**
23. Iterate through a HashMap and print key-value pairs.
24. Find the frequency of each element using a Map.
25. Sort a HashMap by keys and by values. **[asked x3 in your bank]**
26. Sort a collection of employee objects (by ID, name, department, age) using Comparable/Comparator. **[asked x2 in your bank]**

Sorting, Searching & Complexity (1)

27. Sort an array without using built-in methods (Bubble sort). **[asked x3 in your bank]**

Section 2: High Priority (63 questions)

Frequent follow-ups and the next level of difficulty. Practise these once you can do Section 1 without hesitation, with focus on HashMap, Streams and file handling.

String Programs (14)

28. Count the number of words in a string. **[asked x3 in your bank]**
29. Count the occurrences of each word in a sentence (use HashMap) and find duplicate words. **[asked x2 in your bank]**
30. Find the longest substring without repeating characters. **[asked x3 in your bank]**
31. Separate letters, digits, uppercase and lowercase characters from a mixed string (e.g. "aBACbcEDed" to "abcde" and "ABCDE"; "Subbu123raj" to "Subburaj" and "123"). **[asked x3 in your bank]**
32. Compress a string (run-length encoding), e.g. "aabbcccdd" to "a2b2c3d2". **[asked x2 in your bank]**
33. Find all permutations of a string. **[asked x2 in your bank]**
34. Check if one string is a rotation of another. **[asked x2 in your bank]**
35. Swap two strings without a third variable. **[asked x3 in your bank]**
36. Find the longest palindromic substring. **[asked x3 in your bank]**
37. Convert a string to an integer (atoi) and check whether a string contains only digits or is a number (regex). **[asked x5 in your bank]**
38. Convert lowercase to uppercase without using toUpperCase(). **[asked x2 in your bank]**
39. Find the longest and the shortest word in a sentence. **[asked x2 in your bank]**
40. Find the most frequent character in a string.
41. Check for balanced parentheses in a string.

Number & Math Programs (11)

42. Find the sum of digits of a number (loop and recursion). **[asked x5 in your bank]**
43. Check whether a number is an Armstrong number. **[asked x3 in your bank]**
44. Reverse a number.
45. Check whether a number is odd or even; count digits in a number. **[asked x2 in your bank]**
46. FizzBuzz: print "fizz", "buzz" or "fizzbuzz" for multiples of 3, 5 or both.
47. Generate random numbers within a given range.
48. Convert a decimal number to binary, octal and hexadecimal. **[asked x2 in your bank]**
49. Find the GCD and LCM of two numbers.
50. Check whether a number is a perfect number.
51. Check whether a given number is binary. **[asked x2 in your bank]**
52. Check whether a year is a leap year.

Array Programs (13)

53. Find the top two maximum numbers (or the sum of the two largest) in an array. **[asked x4 in your bank]**
54. Print the even numbers from an array; count odd and even numbers. **[asked x3 in your bank]**

55. Find common elements (intersection) between two arrays. **[asked x4 in your bank]**
56. Search for an element in an array. **[asked x2 in your bank]**
57. Sum only the integers in an array that has integers and special characters.
58. Find the non-repeated elements in an array, e.g. [1,1,2,2,3,4,5,5,6,6] gives [3,4]. **[asked x2 in your bank]**
59. Find the first repeating element in an array. **[asked x2 in your bank]**
60. Move all zeroes to the end (or start) of an array, e.g. "32400121200" to "32412120000". **[asked x3 in your bank]**
61. Check whether two arrays are equal (irrespective of order). **[asked x2 in your bank]**
62. Find all pairs of elements whose sum equals a given number.
63. Rotate an array by a given number of steps.
64. Merge two sorted arrays. **[asked x2 in your bank]**
65. Remove a given element from an array; copy an array. **[asked x2 in your bank]**

Collections & HashMap Programs (incl. Test Data Handling) (8)

66. How do you store and retrieve test data dynamically using a Map? How do you handle a key with multiple values? **[asked x2 in your bank]**
67. Convert a HashMap to a List.
68. Sort a list alphabetically, or in reverse order. **[asked x2 in your bank]**
69. Find the first and last element of an ArrayList.
70. Merge two lists and remove duplicates.
71. Identify the first non-repeated element using LinkedHashMap.
72. Remove null or empty values from a collection.
73. Iterate over a list without using loops.

Java 8 Streams Programs (5)

74. Use Java 8 Streams to filter data (e.g. even numbers, test data) and count elements that match a condition. **[asked x3 in your bank]**
75. Transform a list of names to uppercase using Streams.
76. Find the frequency of elements using Streams.
77. Collect stream results into a List or Set, and convert a list to a map. **[asked x2 in your bank]**
78. Remove duplicates from a list using Streams.

Sorting, Searching & Complexity (2)

79. Implement binary search.
80. Implement Selection sort.

OOP, Design Pattern & Exception Coding (7)

81. Write a Singleton class / implement the Singleton design pattern. **[asked x2 in your bank]**
82. Implement hashCode() and equals().
83. Demonstrate method overloading and overriding.
84. Create a simple class with a constructor and getters/setters.

85. Demonstrate interface and abstract class usage.
86. Create a custom exception class.
87. Can we write a method that returns two or more values? How?

File Handling Programs (1)

88. Read a file and print its content; read a text file and count words. **[asked x2 in your bank]**

Multithreading Programs (2)

89. Write a multithreading program that prints odd numbers with one thread and even numbers with another.
90. Write a program to create a deadlock between two threads and explain the deadlock condition. **[asked x2 in your bank]**

Section 3: Medium Priority (45 questions)

Occasionally asked, usually for SDET-leaning roles or as a challenge question. Know the approach even if you cannot write it perfectly.

String Programs (8)

91. Find the length of a string without using length().
92. Reverse a string while preserving the position of spaces.
93. Find all substrings of a given string.
94. Check if a string is a valid shuffle of two strings.
95. Find the longest common prefix.
96. Remove a given character from a string.
97. Print even-indexed characters, or print each letter twice. **[asked x2 in your bank]**
98. Transform 'My name is so and so' into '{My-name-is-so-and-so}'.

Number & Math Programs (4)

99. Find the prime factors of a number.
100. Find the sum of the first 1000 prime numbers.
101. Calculate the difference between two dates in Java.
102. Print patterns and Floyd's Triangle. **[asked x2 in your bank]**

Array Programs (7)

103. Find all the leaders in an integer array.
104. Find the middle index where the sums of both ends are equal.
105. Rearrange an array into alternating positive and negative numbers.
106. Find whether a sub-array with sum equal to zero exists.
107. Find the sub-array with the maximum sum (maximal subarray). **[asked x3 in your bank]**
108. Find the sub-array with the largest product.
109. Find the length of the longest consecutive sequence.

Collections & HashMap Programs (incl. Test Data Handling) (4)

110. How do you handle large data lists efficiently?
111. Reverse traversal of a list using a Stack.
112. Write your own ArrayList implementation.
113. How do you read environment variables dynamically?

Java 8 Streams Programs (3)

114. Sort a list in descending order using Streams.
115. Find the maximum value in a collection using Streams.
116. Merge two lists using Streams.

Sorting, Searching & Complexity (2)

117. How do you calculate the complexity of an algorithm?
118. Sort an array in place using QuickSort (recursive and iterative). **[asked x2 in your bank]**

OOP, Design Pattern & Exception Coding (1)

119. Design a Vending Machine.

File Handling Programs (4)

120. Count characters, words and lines in a text file.
121. Find the word with the highest frequency in a file.
122. Append text to a file.
123. Sort a text file.

Multithreading Programs (2)

124. Write code to avoid deadlock when N threads access N shared resources.
125. Implement the Producer-Consumer pattern using wait, notify and notifyAll.

Linked List, Stack, Matrix & Advanced Problems (8)

126. Find the middle node of a linked list in one pass.
127. Detect whether a linked list contains a cycle.
128. Find the nth element from the end of a linked list.
129. Check whether a linked list is a palindrome.
130. Add two numbers represented by linked lists.
131. Sort a linked list.
132. Sort a Stack using a temporary Stack.
133. Transpose a matrix, add two matrices and multiply matrices. **[asked x3 in your bank]**

Output & Concept Trick Questions (2)

134. What does 1.0 / 0.0 return? Will it throw an exception or a compile-time error?
135. Can you make an array volatile? Can you pass a negative number as an array size? **[asked x2 in your bank]**

Section 4: Low Priority / Quick Revision (16 questions)

Rarely asked for automation testers. Skim the idea, do not spend prep time here before Sections 1 to 3.

String Programs (4)

136. Convert a string to title case.
137. Convert a string to a character array.
138. Replace spaces with %20 (URL encoding).
139. Convert a sentence into an acronym.

Number & Math Programs (5)

140. Find the Roman equivalent of a decimal number.
141. Reverse a number and add until you get a palindrome.
142. Find the largest number less than a given number without a given digit.
143. Find the square root of a number.
144. Find trigonometric values of an angle in Java.

Array Programs (4)

145. Find the minimum value in a rotated sorted array.
146. Find all elements that appear more than n/k times.
147. Find the smallest positive integer that cannot be represented as a subset sum.
148. Find the minimum number of railway platforms required.

Sorting, Searching & Complexity (1)

149. Implement Bucket sort and Counting sort. **[asked x2 in your bank]**

Linked List, Stack, Matrix & Advanced Problems (2)

150. Print a Spiral Matrix.
151. Find shortest-path weights from a source vertex to all vertices in a graph.